



universidad de buenos aires - exactas

departamento de física

laboratorios de enseñanza

Controlador de temperatura para los laboratorios de enseñanza de física

Compendio de I+D, revisiones técnicas y
manual de uso

Marcos Damián Perez

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Licenciatura en Ciencias Físicas

Laboratorios de enseñanza

Buenos Aires, julio 2026

Controlador de temperatura para los laboratorios de enseñanza de física

Compendio de I+D, revisiones técnicas y manual de uso

Marcos Damián Perez

Supervisor: Cobelli, Pablo

Profesor , Departamento de física

Coordinador: Alejandro César Greco

Coordinador, Departamento de física

Pañoleros: Alejandro

Damián

Federico

Juanita

Mateo

Magali

Agustina

Yamila

mas...

Facultad de Ciencias Exactas y Naturales

Departamento de Física

Licenciatura en Ciencias Físicas

Laboratorios de enseñanza

Cuaderno

Buenos Aires, julio 2026

Controlador de temperatura para los laboratorios de enseñanza de física

Copyright © 2026 - Marcos Damián Perez, Facultad de Ciencias Exactas y Naturales.

Este trabajo ... descargo correspondiente.

Agradecimientos

Guía de Escritura

Agradecimientos a quien corresponda.

1

Resumen

Documentación del controlador de temperatura desarrollado para laboratorio 4 de la carrera de licenciatura en física de la universidad de buenos aires. El documento incluye una descripción detallada del diseño, desarrollo y pruebas del controlador, así como un análisis de su desempeño y posibles mejoras futuras. El objetivo principal es proporcionar una herramienta para la medición y control de la temperatura en experimentos de laboratorio, con un enfoque en la precisión, estabilidad y facilidad de uso. El controlador de temperatura desarrollado es capaz de mantener una temperatura constante con una alta precisión, lo que lo hace ideal para una amplia gama de aplicaciones en el laboratorio.

Índice general

1. Resumen	III
<i>Índice de figuras</i>	<i>IX</i>
<i>Índice de cuadros</i>	<i>XI</i>
<i>Glosario</i>	<i>XIII</i>
<i>Siglas</i>	<i>XV</i>
<i>Símbolos</i>	<i>XVII</i>
I Estado Actual del Sistema	1
2. Especificaciones	2
2.1. Actualizacion, 2 de junio de 2026	2
2.2. historial de cambios:	2
3. Arquitectura General del Amplificador	3
3.1. El controlador de temperatura	3
3.1.1. Estado del arte	3
3.1.2. Etapa de salida	4
3.1.3. Medición de corriente y tensión	4
3.1.4. Sistemas de protección	4
3.2. El panel de frontal	4
3.2.1. Controles	4
3.2.2. Indicadores	4
3.2.3. Display	4
3.3. Interfaz de control remoto y programación	4
3.3.1. Interfaces de comunicación	4
3.3.2. Protocolos de comunicación	4
3.3.3. Comandos SCPI	4
4. Controlador-de-Temperatura	5
5. Control y Medición	6

6. Protecciones	7
7. Alimentación	8
8. Interfaz de Usuario	9
9. Diseño Térmico	10
10. Diseño Mecánico	11
11. Integración Electromecánica	12
12. Implementacion Electronica	13
12.1. Esquematicos	13
12.2. PCBs	13
13. Implementacion Mecanica	14
14. Implementación de Firmware	15
14.1. dependencias	15
15. Validación Eléctrica	16
16. Validación Térmica	17
17. Validación del Sistema	18
18. Manual de Operación	19
19. Mantenimiento	20
 II Gestión de Versiones del Producto	 21
20. Historial de Versiones	22
 III Desarrollo de Firmware	 23
21. Desarrollo de firmware	24
21.1. Resumen	24
22. Sistema de Comunicación	25
22.1. Arquitectura General	25
22.1.1. Modelo de Capas y APIs	25
22.1.2. APIs del sistema de comunicación	27
22.1.3. Principio de Propiedad y Gestión de Contención	27
22.1.4. Eventos: Origen y Jerarquía	27

22.1.5. Flujo de Información entre Capas	28
22.2. Capa de Driver	28
22.3. Capa de Middleware	29
22.3.1. Communication Task	29
22.3.2. Modelo de Ejecución y Planificación	29
22.3.3. Registro de Eventos y Captura Atómica (Snapshot)	30
22.3.4. Orden de Precedencia y Flujo de Control	30
22.3.5. Máquinas de Estado de Transporte	30
22.3.6. Adaptación a Protocolos de Instrumentación	32
22.3.7. Inicialización del Subsistema de Comunicación	33
22.4. Capa de Aplicación	33
22.5. API de Driver (<code>comm_iface_t</code>)	34
22.5.1. Interfaz Abstracta <code>comm_iface_t</code>	34
22.5.2. Estados de la Interfaz	34
22.5.3. Eventos de Comunicación y Error	34
22.5.4. APIs de Control de Flujo	35
22.5.5. Protección de Acceso Concurrente	36
22.5.6. Estrategias de Sincronización y Gestión de Buffers	36
22.6. API de Sistema (<code>comm_sys_api</code>)	36
22.6.1. Estados del Subsistema	36
22.6.2. Comandos del Sistema	37
22.6.3. Eventos de Notificación	37
22.6.4. Funciones de Control	38
22.6.5. Funciones de Consulta	38
22.6.6. Notificaciones Asíncronas	38
22.7. API de Aplicación (<code>app_msg_iface_t</code>)	38
22.7.1. Interfaz <code>app_msg_iface_t</code>	39
22.7.2. Callbacks de Recepción	39
22.7.3. Callbacks de Transmisión	39
22.7.4. Callbacks de Error	39
22.7.5. Eventos de Aplicación	39
22.7.6. Errores de Aplicación	39
22.7.7. Envío de Respuestas	40
22.7.8. Códigos de Retorno	40
23. Integración del intérprete SCPI con el sistema	41
23.1. Integración con la capa de <i>middleware</i>	41
23.1.1. Recepción de bytes crudos	41
23.1.2. Transmisión de respuestas	42
23.1.3. Notificación de errores de comunicación	42
23.1.4. Secuencia de inicialización	42
23.2. Interacción con el sistema	42

23.2.1. Comandos que afectan al estado global	43
23.2.2. Recepción de eventos del sistema	43
23.3. Interacción con los subsistemas de control	43
23.3.1. API de sensor	44
23.3.2. API de controlador PID	44
23.3.3. API de salida	45
23.3.4. Mapeo de comandos SCPI a las APIs de control	45
23.4. Consideraciones sobre la implementación actual	45

Anexos

Índice de figuras

22.1. Modelo de capas del sistema de comunicación. Las flechas indican las dependencias funcionales y el sentido del flujo de datos.	25
--	----

Índice de cuadros

22.1. Propiedad de recursos en el subsistema de comunicación.	28
22.2. Tabla de transiciones de la FSM de Gestión de Interfaz.	31
22.3. Tabla de transiciones de la FSM de Transmisión.	32
22.4. Estados definidos en <code>comm_iface_state_t</code>	35
22.5. Valores definidos en <code>comm_response_t</code>	35
22.6. Estados del subsistema definidos en <code>comm_sys_state_t</code>	37
22.7. Comandos definidos en <code>comm_sys_cmd_t</code>	37
22.8. Eventos de notificación definidos en <code>comm_sys_event_t</code>	37
22.9. Callbacks definidos en <code>app_msg_iface_t</code>	39
22.10. Eventos definidos en <code>app_msg_event_t</code>	40
22.11. Errores definidos en <code>app_msg_error_t</code>	40
22.12. Valores definidos en <code>app_msg_response_t</code>	40
23.1. Funciones de la API de sensor.	44
23.2. Funciones de la API de PID.	44
23.3. Funciones de la API de salida.	45
23.4. Correspondencia entre comandos SCPI y APIs de control.	46

Parte I

Estado Actual del Sistema

2

Especificaciones

2.1 Actualizacion, 2 de junio de 2026

Especificaciones del sistema en desarrollo. Estas especificaciones pueden ser modificadas a lo largo del proyecto, pero es importante tener una base clara desde el principio para guiar el diseño y desarrollo del controlador de temperatura.

- Seguridad: El diseño debe cumplir con las normativas de seguridad del laboratorio, evitando riesgos para los estudiantes y el equipo.
- Corriente máxima de salida: 10 A continuos (DC).
- Tensión máxima de salida: 24 V.
- Ancho de banda: DC a 10 Hz.
- Precisión en DC: offset menor a 5 mV (si puede ser menos, mejor).
- Ruido de salida menor a 10 mV
- Sistemas de protección: El diseño debe incluir protecciones contra sobrecorriente, sobretensión, cortocircuitos y sobrecalentamiento para garantizar la seguridad y la durabilidad del equipo.
- Interfaz de control: El controlador debe ser controlable a través de una interfaz digital (por ejemplo, USB, Ethernet o RS-232) para permitir su integración con sistemas de adquisición de datos y controladores externos.
- El equipo debe soportar comandos SCPI para facilitar su integración con software de control y adquisición de datos comúnmente utilizado en laboratorios.
- Interfaz de usuario: El Controlador debe contar con una interfaz de usuario intuitiva para facilitar su operación por parte de los estudiantes y el personal del laboratorio, incluyendo indicadores de estado y controles de ajuste de la salida.

2.2 historial de cambios:

- **2 de junio del 2026:** Se establecieron las especificaciones del sistema en desarrollo.

3

Arquitectura General del Controlador de temperatura

En esta sección se presenta una descripción general de la arquitectura del amplificador lineal de potencia, destacando los componentes principales y su función dentro del sistema. Se abordarán aspectos como la etapa de entrada, la etapa de amplificación, la etapa de salida y los sistemas de protección y control.

3.1 El controlador de temperatura

Actualmente para no hay en el alboratorio un controlador de temperatura y solo se deja evolucionar la temperatura midiendo en los trancitorios y estacionarios. El objetivo de este proyecto es desarrollar un controlador de temperatura que permita mantener una temperatura constante con alta precisión, lo que es fundamental para una amplia gama de experimentos en el laboratorio, no solo para tener la bariable controlada sino tambien para acelerar los largo transitorios de termalización de los experimentos. El controlador de temperatura desarrollado se integrará con el sistema de adquisición de datos del laboratorio, permitiendo un control preciso y una fácil monitorización de la temperatura durante los experimentos.

3.1.1 Estado del arte

- Etapa de salida: Puentes H con transistores MOSFET de potencia en configuración push-pull para manejar las altas corrientes y tensiones requeridas, con un diseño que minimice las pérdidas de conmutación y maximice la eficiencia.
- Control de la etapa de salida: modulación por ancho de pulso (PWM) para controlar la potencia entregada a la carga, con un controlador de retroalimentación para mantener la temperatura constante.
- Filtro de salida: Un filtro LC para suavizar la señal de salida y reducir el ruido, asegurando una entrega de potencia estable y precisa a la carga.

- Control y medición: Un microcontrolador o FPGA para implementar el control del amplificador, la adquisición de datos de los sensores de corriente y tensión, y la comunicación con la interfaz de usuario.
- Sistemas de protección: Circuitos de protección contra sobrecorriente, sobretensión, cortocircuitos y sobrecalentamiento para garantizar la seguridad y la durabilidad del equipo.
- Medición de corriente y tensión: sensores de corriente y tensión de alta precisión para monitorear la salida del amplificador y proporcionar retroalimentación al controlador.
- Interfaz de usuario: Pantalla LCD o LED para mostrar el estado del amplificador, controles de ajuste para configurar la salida y botones para encender/apagar el dispositivo.

3.1.2 Etapa de salida

3.1.3 Medición de corriente y tensión

3.1.4 Sistemas de protección

3.2 El panel de frontal

3.2.1 Controles

3.2.2 Indicadores

3.2.3 Display

3.3 Interfaz de control remoto y programación

3.3.1 Interfaces de comunicación

3.3.2 Protocolos de comunicación

3.3.3 Comandos SCPI

4

Controlador-de-Temperatura

5

Control y Medición

6

Protecciones

7

Alimentación

8

Interfaz de Usuario

9

Diseño Térmico

10

Diseño Mecánico

11

Integración Electromecánica

12

Implementación electrónica

12.1 Esquematicos

12.2 PCBs

13

Implementación mecánica

14

Implementacion de Firmware

14.1 dependencias

-

15

Validación Eléctrica

16

Validación Térmica

17

Validación del Sistema

18

Manual de Operación

19

Mantenimiento

Parte II

Gestión de Versiones del Producto

20

Historial de Versiones

Parte III

Desarrollo de Firmware

21

Desarrollo de firmware

21.1 Resumen

Estructura y organización y resumen del desarrollo:

Sistema de comunicación: Driver Dependencias y obligaciones API Inyección de dependencias (buffers, etc) Communication Task Mantenimiento de interfaz Manejo de errores Recepción y extracción de mensajes Transmisión API Motor SCPI Búsqueda de comandos Parser SCPI Control de errores Armado de Respuestas Escritura de mensajes de respuesta

Panel Frontal: Pantalla API Control API

Control: Salida de potencia Seguridad API Control PID Modelo incremental para DSP Sensores de temperatura Filtros API

22

Sistema de Comunicación

22.1 Arquitectura General

El subsistema de comunicación se organiza en tres capas funcionales que intercambian información exclusivamente a través de contratos abstractos bien definidos. Esta separación permite modificar o reemplazar cualquiera de las capas sin afectar a las demás, siempre que se respeten las interfaces establecidas.

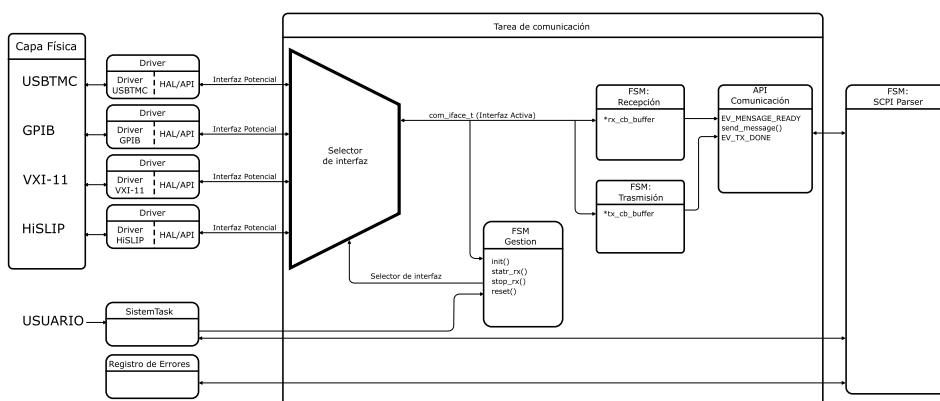


Figura 22.1: Modelo de capas del sistema de comunicación. Las flechas indican las dependencias funcionales y el sentido del flujo de datos.

22.1.1 Modelo de Capas y APIs

Capa de Driver (Interfaz de Transporte Físico) Constituye el estrato más próximo al hardware. Su responsabilidad exclusiva es la gestión del periférico de comunicación (UART, USBTMC, SPI, TCP/IP, etc.) y la traducción de sucesos físicos a una representación abstracta normalizada. Cada driver implementa la interfaz `comm_iface_t`, que constituye el único punto de contacto con la capa de middleware. Las responsabilidades de esta capa comprenden:

- La administración de los recursos de hardware (registros, interrupciones, canales DMA).

- La producción de eventos abstractos de interfaz (dato recibido, transmisión completada, error).
- El mantenimiento del estado operativo de la conexión física.
- La operación sobre los buffers de transporte inyectados, sin asignar buffers propios ni procesar el contenido de los datos transportados.

Capa de Middleware La capa de middleware constituye el núcleo lógico del subsistema de comunicación. Su elemento central es la *Communication Task*, una tarea independiente que opera sobre la interfaz abstracta proporcionada por el driver y coordina las máquinas de estado de gestión, recepción y transmisión. Las responsabilidades de esta capa abarcan:

- La gestión global del ciclo de vida de la interfaz (inicialización, arranque, detención, recuperación ante fallos).
- La recepción de bytes desde el driver y la delimitación de mensajes completos de acuerdo con las reglas del protocolo de instrumentación (por ejemplo, detección del terminador LF según IEEE 488.2).
- La transmisión de mensajes hacia el driver cuando la aplicación lo solicita.
- La exposición de servicios hacia el resto del sistema mediante la `comm_sys_api` y hacia la aplicación mediante la `app_msg_iface_t`.

La capa de middleware desconoce por completo el contenido semántico de los mensajes que transporta. Para esta capa, un mensaje es una secuencia de bytes delimitada por un terminador; no realiza análisis sintáctico ni interpreta comandos.

Configuración e inicialización. La capa de middleware no se autoinicializa. Recibe los comandos de inicio, parada y cambio de interfaz a través de la `comm_sys_api`, y es el módulo de sistema (`task_system`) quien emite dichos comandos tras registrar las interfaces disponibles. La selección de la interfaz activa es realizada por la *Communication Task* en base a la configuración persistente del instrumento y a las solicitudes externas recibidas.

Capa de Aplicación Es el estrato superior, consumidor de los servicios de comunicación. Recibe mensajes completos desde la capa de middleware y los procesa de acuerdo con el protocolo de aplicación elegido (SCPI, Modbus, propietario, etc.). Sus responsabilidades son:

- Interpretar el contenido de los mensajes y ejecutar las acciones solicitadas.
- Generar respuestas conformes al protocolo correspondiente.
- Interactuar con el resto del sistema del instrumento (control, medición, configuración).

La capa de aplicación no accede directamente al hardware de comunicación ni a los buffers de transporte. Recibe los mensajes a través de la interfaz `app_msg_iface_t` y envía sus respuestas mediante la función `comm_app_send_response()` que ofrece la capa de middleware.

22.1.2 APIs del sistema de comunicación

La arquitectura del subsistema de comunicación define tres interfaces abstractas que vinculan las capas entre sí y con el resto del sistema:

comm_iface_t (API de Driver): Contrato entre la capa de middleware y cualquier periférico físico. Define callbacks de ciclo de vida, control de flujo, gestión de eventos y protección concurrente. Se describe en detalle en la sección 22.2.

comm_sys_api (API de Sistema): Permite a los módulos de sistema y de interfaz de usuario controlar el subsistema de comunicación (iniciar, detener, cambiar de medio) y consultar su estado. Se detalla en la sección 22.6.

app_msg_iface_t (API de Aplicación): Interfaz que la capa de aplicación debe implementar para recibir mensajes y notificaciones desde la capa de middleware. Se describe en la sección 22.7.

22.1.3 Principio de Propiedad y Gestión de Contención

Cada capa es propietaria exclusiva de los recursos que administra. Este principio no elimina la contención —inherente a un sistema con productores y consumidores asíncronos—, sino que la hace explícita y la controla mediante mecanismos bien definidos: secciones críticas acotadas, capturas atómicas (*snapshot*) y referencias inyectadas.

- Los buffers de transporte (recepción y transmisión) son propiedad de la *Communication Task* y se inyectan al driver por referencia durante la inicialización. El driver opera sobre ellos pero no los posee.
- El contexto del periférico (registros de configuración, estado interno del driver) es privado de cada implementación concreta del driver y no es accesible desde fuera.
- Los buffers de mensajes y respuestas pertenecen a la capa de aplicación y son independientes del transporte.

La tabla 22.1 detalla la asignación de propiedad para cada recurso compartido en el subsistema de comunicación.

22.1.4 Eventos: Origen y Jerarquía

El subsistema de comunicación distingue tres categorías de eventos según su origen:

Eventos de Interfaz (comm_iface_event_t): Generados por el driver a partir de sucesos de hardware. Incluyen notificaciones de flujo normal (IFACE_EVENT_RX_DATA_AVAILABLE, IFACE_EVENT_TX_COMPLETE) y de error. Se consumen exclusivamente en la capa de middleware mediante el mecanismo de *snapshot*.

Cuadro 22.1: *Propiedad de recursos en el subsistema de comunicación.*

Recurso	Propietario	Observación
Buffer RX de transporte	Communication Task	Inyectado en <code>comm_iface_t</code>
Buffer TX de transporte	Communication Task	Inyectado en <code>comm_iface_t</code>
Contexto del periférico	Driver	Privado de cada implementación
Registro de eventos	Driver	Compartido ISR/Task mediante s
Estado de interfaz	Driver	Visible en modo lectura para capa
Registro de interfaces	Infraestructura del subsistema de comunicación	<code>comm_iface_registry</code>
Comandos del sistema	<code>comm_sys_api</code>	Consumidos atómicamente por la
Buffers de mensajes	Capa de aplicación	Independientes del transporte
Buffers de respuesta	Capa de aplicación	Independientes del transporte
Configuración persistente	<code>config_manager</code>	Interfaz preferida, parámetros PII

Eventos Lógicos del Middleware: Producidos por la *Communication Task* al completar una operación de alto nivel. Los principales son `EV_MESSAGE_READY` (mensaje delimitado disponible) y `EV_TX_DONE` (transmisión finalizada). Se entregan a la aplicación a través de `app_msg_iface_t`.

Comandos del Sistema (`comm_sys_cmd_t`): Publicados por módulos externos (sistema, interfaz de usuario) mediante `comm_sys_api`. Se consumen atómicamente al inicio del ciclo de la *Communication Task*, de forma análoga a los eventos de interfaz.

Esta jerarquía asegura que cada tipo de evento se procese en la capa adecuada y que los estratos superiores reciban información ya filtrada y pertinente para su función.

22.1.5 Flujo de Información entre Capas

La comunicación entre capas sigue un modelo orientado a eventos, sin llamadas bloqueantes. El flujo de información se articula en tres direcciones:

Recepción: Medio físico → Driver → Eventos de interfaz → Communication Task → Mensaje completo → Capa de Aplicación.

Transmisión: Capa de Aplicación → `comm_app_send_response()` → Communication Task → Driver → Medio físico.

Control: Sistema / Interfaz de Usuario → `comm_sys_api` → Communication Task → Driver.

La capa de aplicación se describe en las secciones siguientes de este mismo capítulo. El capítulo siguiente detalla la implementación concreta de un analizador SCPI empleando la biblioteca `libscpi`, en conformidad con el estándar IEEE 488.2.

22.2 Capa de Driver

La capa de driver constituye el punto de contacto entre el hardware de comunicación y el firmware de nivel superior. Su responsabilidad exclusiva es la gestión del

periférico físico (UART, USBTMC, TCP/IP, etc.) y la traducción de eventos de hardware a una representación abstracta normalizada. Cada driver implementa la interfaz `comm_iface_t`, que constituye el único mecanismo de interacción con la capa de middleware. La implementación concreta de cada driver es transparente para las capas superiores, que operan exclusivamente sobre los callbacks definidos en dicha interfaz.

22.3 Capa de Middleware

La capa de middleware constituye el núcleo lógico del subsistema de comunicación. Su elemento central es la *Communication Task*, una tarea independiente que opera sobre la interfaz abstracta proporcionada por el driver y coordina las máquinas de estado de gestión, recepción y transmisión. Esta capa es responsable de la delimitación de mensajes, la gestión del ciclo de vida de la interfaz y la exposición de servicios hacia el sistema y la aplicación.

22.3.1 Communication Task

La *Communication Task* es el núcleo de ejecución del middleware. Se implementa como un bucle cooperativo que, en cada iteración, ejecuta secuencialmente las tres máquinas de estado: Gestión, Recepción y Transmisión. Su diseño es independiente del hardware subyacente y del protocolo de aplicación, operando exclusivamente sobre las interfaces abstractas `comm_iface_t` (hacia el driver) y `app_msg_iface_t` (hacia la aplicación).

22.3.2 Modelo de Ejecución y Planificación

La tarea de comunicación está diseñada para ejecutarse en un entorno cooperativo, ya sea como un hilo independiente dentro de un RTOS o como un bucle dentro de un esquema *super-loop* en sistemas *bare-metal*. El modelo de ejecución respeta los siguientes principios:

- **Ejecución no bloqueante:** cada FSM evalúa sus entradas y retorna el control sin esperas activas. Si una operación no puede completarse, se difiere al siguiente ciclo.
- **Temporización simple:** los tiempos de espera se gestionan mediante una única base de tiempo del sistema (`sys_tick`) con resolución de milisegundos.
- **Determinismo:** la ejecución secuencial de las FSMs en un único contexto elimina por diseño las condiciones de carrera internas. La única sección crítica que requiere protección externa es la captura del *snapshot* de eventos, resuelta atómicamente en el driver.

22.3.3 Registro de Eventos y Captura Atómica (Snapshot)

El driver mantiene un registro de eventos en forma de máscara de bits (`comm_iface_event_t`) que es escrito de manera asíncrona por la ISR o por las propias funciones del driver. Para operar sobre una base coherente y sin condiciones de carrera, la *Communication Task* implementa el siguiente procedimiento al inicio de cada ciclo:

1. **Captura atómica:** se invoca el callback `get_event()` de la interfaz activa. Dentro de una sección crítica, se lee el valor actual del registro y se pone a cero (*clear-on-read*). El valor leído se almacena en una variable local denominada *snapshot de eventos*.
2. **Procesamiento sobre copia local:** durante el resto del ciclo, ninguna FSM accede al registro físico. Todas trabajan sobre el *snapshot*, aplicando máscaras binarias para aislar los eventos de su competencia.
3. **Nuevos eventos:** los eventos de hardware ocurridos durante el ciclo se escriben en el registro del driver (ahora vacío) y serán capturados en el *snapshot* del ciclo siguiente.

Este mecanismo garantiza la coherencia temporal: todas las FSM evalúan exactamente el mismo estado del hardware para un mismo ciclo de ejecución.

22.3.4 Orden de Precedencia y Flujo de Control

El *snapshot* de eventos es evaluado por las FSM en un orden fijo que refleja la criticidad de cada función:

1. **FSM de Gestión de Interfaz (prioridad máxima):** procesa eventos de cambio de conexión y errores críticos. Puede alterar el estado global antes de que las demás máquinas evalúen sus eventos.
2. **FSM de Recepción (RX):** procesa los eventos `IFACE_EVENT_RX_DATA_AVAILABLE` y errores de recepción, extrayendo bytes del buffer circular y delimitando mensajes.
3. **FSM de Transmisión (TX):** procesa eventos de finalización de envíos y errores de transmisión. También atiende solicitudes de transmisión generadas internamente.

El flujo macroscópico del bucle operativo es, por tanto:

Snapshot → FSM Gestión → FSM RX → FSM TX

22.3.5 Máquinas de Estado de Transporte

Las tres máquinas de estado que gobiernan el comportamiento del middleware se modelan según el formalismo de Mealy: las transiciones y las acciones dependen del estado actual y de los eventos de entrada. Los estados son estrictamente privados y

la comunicación entre máquinas se realiza mediante la publicación de eventos lógicos internos.

FSM de Gestión de Interfaz

Esta máquina supervisa el estado global de la interfaz, coordina las transiciones operativas, la recuperación ante fallos y la selección de un nuevo medio de transporte cuando es necesario.

Cuadro 22.2: Tabla de transiciones de la FSM de Gestión de Interfaz.

Estado Actual	Evento	Siguiente Estado	Acción
DOWN	CMD_START	READY	init()
READY	IFACE_EVENT_INTERFACE_CONNECTED	ACTIVE	start_rx()
ACTIVE	CMD_DISABLE_RX	READY	stop_rx()
ACTIVE	Evento de error crítico	ERROR	Publicar condición de falla
ERROR	CMD_RESET	READY	reset()
ERROR	Error irreparable / cambio requerido	SELECTING	deinit() actual; invocar selector
SELECTING	Interfaz disponible	READY	init() sobre nueva interfaz
SELECTING	Sin interfaces disponibles	DOWN	Notificar indisponibilidad

Cuando la FSM transita a SELECTING, ejecuta su lógica interna de selección: consulta el registro de interfaces disponibles, evalúa la interfaz preferida configurada en el sistema y determina la nueva interfaz activa. Si hay una interfaz disponible, transita a READY e invoca `init()` sobre ella. Si no hay ninguna, retorna a DOWN y notifica la indisponibilidad del subsistema.

La FSM también atiende solicitudes de cambio de interfaz provenientes de la API de sistema. Cuando un módulo externo solicita un cambio, la FSM evalúa la viabilidad y, si es posible, inicia la transición a SELECTING.

Los comandos del sistema se reciben a través de un registro de comandos que la FSM consume atómicamente al inicio de cada ciclo, de forma análoga al *snapshot* de eventos de la interfaz.

FSM de Recepción (RX)

La FSM de Recepción transfiere los bytes recibidos desde el buffer circular `rx_buffer` directamente a la capa de aplicación, sin realizar delimitación ni interpretación alguna. En cada ciclo de la *CommunicationTask*, se protege el buffer de recepción, se extraen todos los bytes disponibles mediante `cb_get()` y se entregan a la aplicación a través del callback `on_rx_data` de la interfaz `app_msg_iface_t`. Una vez transferidos los datos, se libera la protección.

Esta estrategia elimina la necesidad de un buffer intermedio de mensajes en el middleware y delega la delimitación de comandos al parser de la capa de aplicación (por ejemplo, `libscpi`), que gestiona internamente su propio buffer de entrada y los terminadores configurados.

FSM de Transmisión (TX)

Esta máquina administra el flujo de salida de datos. Responde a las solicitudes de transmisión de la capa de aplicación y gestiona el envío de los datos acumulados en `tx_buffer` hacia el driver.

Cuadro 22.3: Tabla de transiciones de la FSM de Transmisión.

Estado Actual	Evento	Siguiente Estado	Acción
TX_IDLE	EV_TX_REQUEST	TX_BUSY	Invocar <code>start_tx()</code>
TX_BUSY	COMM_IFACE_OK	TX_BUSY	Esperar TX_COMPLETE
TX_BUSY	COMM_IFACE_BUSY	TX_BUSY	Reintentar en siguiente ciclo
TX_BUSY	COMM_IFACE_IDLE	TX_IDLE	Sin datos pendientes
TX_BUSY	COMM_IFACE_ERROR	TX_IDLE	Publicar error de TX
TX_BUSY	IFACE_EVENT_TX_COMPLETE	TX_IDLE	Publicar EV_TX_DONE
TX_BUSY	IFACE_EVENT_TX_ERROR_...	TX_IDLE	Publicar error de TX; la FSM de Gestión ejec

La máquina evalúa el código de retorno de `start_tx()` para decidir la acción inmediata, sin consultar el estado de la interfaz. Los eventos de finalización y error se reciben a través del *snapshot* en ciclos posteriores.

22.3.6 Adaptación a Protocolos de Instrumentación

La capa de middleware es responsable de la delimitación de mensajes de acuerdo con las reglas del protocolo de instrumentación aplicable, pero no interpreta su contenido.

Delimitación de mensajes Para protocolos orientados a texto, la FSM de Recepción identifica el fin de mensaje al detectar el carácter LF (0x0A). El mensaje completo, incluyendo cualquier carácter de control precedente como CR, se entrega íntegramente a la capa de aplicación. La interpretación del contenido recae exclusivamente en el parser de la capa superior, que puede optar por descartar o procesar dichos caracteres según el estándar aplicable (IEEE 488.2, en el caso de SCPI).

Independencia del protocolo La capa de middleware no distingue entre un comando SCPI, una consulta de estado o cualquier otra carga útil. Opera exclusivamente con mensajes entendidos como secuencias de bytes delimitadas. Esta independencia permite que el mismo código de middleware dé servicio a distintos protocolos de aplicación sin más cambio que el motor de procesamiento conectado a su API.

La implementación actual del middleware está configurada para protocolos basados en líneas de texto (ASCII) que emplean el carácter LF como terminador. Esta estrategia es adecuada para SCPI y otros protocolos conformes a IEEE 488.2. Para protocolos binarios como Modbus RTU, que delimitan tramas mediante intervalos de silencio, o para protocolos con terminadores diferentes, la FSM de Recepción requeriría una modificación en su lógica de delimitación. Esta extensión no afecta a la arquitectura general

del subsistema, ya que las APIs de Driver y de Aplicación permanecen invariantes.

Interrupciones de consulta (*Query Interrupts*) Si el protocolo del periférico subyacente dispone de mecanismos de interrupción de consulta —como el control de flujo en USBTMC—, el driver traduce dicha condición de hardware en un evento abstracto de interfaz. La FSM de Gestión captura este evento y coordina la acción correspondiente evaluando el estado operativo actual, sin que el significado del comando trascienda a esta capa.

22.3.7 Inicialización del Subsistema de Comunicación

El arranque del subsistema de comunicación sigue una secuencia bien definida, orquestada por el módulo de sistema (`task_system`):

1. **Carga de configuración:** el sistema carga los valores por defecto y luego intenta leer la configuración persistente desde memoria no volátil mediante `config_manager`. Entre los parámetros recuperados se encuentra la interfaz de comunicación preferida.
2. **Registro de interfaces:** se invocan las funciones de registro de cada driver disponible, inyectando los buffers de transporte que residen en la *Communication Task*. Cada driver se agrega al registro central (`comm_iface_registry`).
3. **Inicio de la comunicación:** una vez registrada al menos una interfaz, el sistema publica el comando `COMM_SYS_CMD_START` a través de `comm_sys_start()`.
4. **Reacción de la FSM de Gestión:** en el siguiente ciclo de la *Communication Task*, la FSM consume el comando, invoca al selector interno —que consulta `config_manager` para conocer la interfaz preferida—, ejecuta `init()` sobre la interfaz elegida y transita a `READY`.
5. **Activación de la recepción:** al recibir el evento `IFACE_EVENT_INTERFACE_CONNECTED`, la FSM invoca `start_rx()` y transita a `ACTIVE`. A partir de este momento, la FSM de Recepción comienza a delimitar mensajes.

Este flujo garantiza que los buffers de transporte estén correctamente inyectados, que la configuración del usuario sea respetada y que la comunicación arranque sin intervención manual.

22.4 Capa de Aplicación

La capa de aplicación constituye el estrato superior del subsistema de comunicación. Consume los mensajes completos provistos por la capa de middleware y los procesa de acuerdo con la sintaxis y semántica del protocolo de aplicación elegido (SCPI, Modbus, propietario, etc.). Sus responsabilidades incluyen la interpretación del contenido de los mensajes, la ejecución de las acciones solicitadas sobre el instrumento y la generación de respuestas conformes al protocolo correspondiente.

Esta capa no accede directamente al hardware de comunicación ni a los buffers de transporte. Interactúa con la capa de middleware exclusivamente a través de la interfaz `app_msg_iface_t` (para recibir mensajes y notificaciones) y de la función `comm_app_send_response()` (para enviar respuestas). El capítulo siguiente describe en detalle la implementación de un intérprete SCPI como caso particular de capa de aplicación.

22.5 API de Driver (`comm_iface_t`)

La API de Driver establece el contrato vinculante entre la capa de middleware y cualquier periférico de comunicación físico. Se define en la estructura `comm_iface_t`, que todo driver debe implementar para ser utilizable por la capa de middleware.

22.5.1 Interfaz Abstracta `comm_iface_t`

La estructura `comm_iface_t` contiene los siguientes grupos funcionales:

- **Identificación:** campo `id` (`comm_iface_id_t`) que identifica unívocamente la interfaz y campo `name` para diagnóstico.
- **Contexto privado:** puntero opaco `void *context` que cada implementación utiliza para almacenar sus datos internos. No es accedido por las capas superiores.
- **Estado y eventos:** campos `state` y `event` que mantienen la condición operativa actual y el registro de eventos pendientes.
- **Buffers de transporte inyectados:** punteros `rx_buffer` y `tx_buffer` a sendos buffers circulares, propiedad de la *Communication Task* e inyectados antes de `init()`.
- **Callbacks de ciclo de vida:** `init()`, `deinit()` y `reset()`. Retornan `comm_error_t`.
- **Callbacks de control de flujo:** `start_rx()`, `stop_rx()` y `start_tx()`. Retornan `comm_response_t`.
- **Callback de gestión de eventos:** `get_event()`. La escritura de eventos es responsabilidad exclusiva del driver y se realiza de forma interna.
- **Callbacks de protección concurrente:** `protect_rx()`, `unprotect_rx()`, `protect_tx()` y `unprotect_tx()`.

22.5.2 Estados de la Interfaz

El estado operativo de una interfaz se representa mediante el campo `state`, una máscara de bits de tipo `comm_iface_state_t`.

Los estados son condiciones internas del driver, modificadas exclusivamente por él. Las capas superiores no acceden a ellos; la capa de middleware evalúa el resultado de las operaciones invocadas sin consultar el estado.

22.5.3 Eventos de Comunicación y Error

Los eventos son sucesos instantáneos notificados por el driver a la *Communication-Task* mediante el campo `event` (`comm_iface_event_t`).

Cuadro 22.4: Estados definidos en *comm_iface_state_t*.

Símbolo	Máscara	Significado
COMM_STATE_NONE	0x00	Sin estado activo.
COMM_STATE_CONNECTED	0x01	Enlace físico o lógico establecido.
COMM_STATE_RX_ACTIVE	0x02	Recepción habilitada.
COMM_STATE_TX_ACTIVE	0x04	Transmisión en curso.
COMM_STATE_ERROR	0x08	Condición de falla global.

Eventos de Flujo Normal

IFACE_EVENT_INTERFACE_CONNECTED: Interfaz operativa.

IFACE_EVENT_INTERFACE_DISCONNECTED: Conexión perdida.

IFACE_EVENT_RX_DATA_AVAILABLE: Datos disponibles en buffer de recepción.

IFACE_EVENT_TX_COMPLETE: Transmisión finalizada.

Eventos de Error

Recepción: IFACE_EVENT_RX_ERROR_OVERFLOW, IFACE_EVENT_RX_ERROR_FRAMING, IFACE_EVENT_RX_ERROR_OTHER.

Transmisión: IFACE_EVENT_TX_ERROR_TIMEOUT, IFACE_EVENT_TX_ERROR_BUS_FAULT.

Interno: IFACE_EVENT_INTERNAL_ERROR.

La activación de un evento de error puede provocar adicionalmente la activación del estado COMM_STATE_ERROR. La escritura de eventos se realiza directamente en el campo event bajo la protección adecuada.

22.5.4 APIs de Control de Flujo

Los callbacks de control de flujo son no bloqueantes y retornan *comm_response_t*.

Cuadro 22.5: Valores definidos en *comm_response_t*.

Símbolo	Valor	Interpretación
COMM_IFACE_OK	0x00	Operación exitosa.
COMM_IFACE_ERROR	0x01	Error.
COMM_IFACE_BUSY	0x02	Ocupado; reintentar.
COMM_IFACE_IDLE	0x03	Sin datos pendientes (<i>start_tx</i>).

El tipo *comm_response_t* es compartido por la API de Driver y la API de Sistema para todas las operaciones que retornan un resultado inmediato. Los valores COMM_IFACE_OK, COMM_IFACE_ERROR, COMM_IFACE_BUSY e COMM_IFACE_IDLE constituyen el conjunto completo de códigos de retorno del subsistema de comunicación.

start_rx() Activa el canal de recepción. Retorna COMM_IFACE_OK o COMM_IFACE_ERROR.

stop_rx() Detiene la recepción. Retorna COMM_IFACE_OK o COMM_IFACE_ERROR.

start_tx() Inicia la transmisión de datos desde `tx_buffer`. Evalúa el estado del hardware y la disponibilidad de datos, retornando el código correspondiente.

22.5.5 Protección de Acceso Concurrente

Los buffers de recepción y transmisión son recursos compartidos entre la *Communication Task* y la ISR/DMA. La interfaz proporciona cuatro callbacks de protección: `protect_rx()`, `unprotect_rx()`, `protect_tx()` y `unprotect_tx()`. La implementación concreta depende del periférico y del entorno de ejecución (deshabilitación de interrupciones en *bare-metal*, mutex en RTOS, etc.).

22.5.6 Estrategias de Sincronización y Gestión de Buffers

Los buffers de recepción y transmisión son buffers circulares propiedad de la *Communication Task*. El driver accede a ellos exclusivamente para escribir bytes recibidos (RX) o leer bytes a transmitir (TX). Se describen dos estrategias:

Acceso Directo al Buffer Circular

La ISR o el DMA operan directamente sobre el buffer circular inyectado. La detección de desbordamiento en recepción genera el evento `IFACE_EVENT_RX_ERROR_OVERFLOW`.

Doble Buffer Interno del Driver

Cuando el periférico no permite acceso DMA directo, el driver implementa un doble buffer interno, transparente para la *Communication Task*. Los datos se copian entre los buffers internos y el buffer circular bajo la protección adecuada.

22.6 API de Sistema (`comm_sys_api`)

La API de Sistema constituye el contrato entre el subsistema de comunicación y los módulos externos que requieren controlarlo o consultar su estado. A través de esta interfaz, el módulo de sistema, la interfaz de usuario o cualquier otro componente del firmware puede iniciar, detener y reiniciar la comunicación, seleccionar el medio de transporte activo, consultar el estado global y registrarse para recibir notificaciones asíncronas.

Todas las funciones de esta API son no bloqueantes. Las operaciones de control no ejecutan acciones directas sobre el hardware, sino que publican comandos en un registro interno que la *Communication Task* consume atómicamente al inicio de cada ciclo, de forma análoga al mecanismo de *snapshot* empleado para los eventos de la interfaz.

22.6.1 Estados del Subsistema

El estado global del subsistema de comunicación se representa mediante el tipo enumerado `comm_sys_state_t`. Este estado refleja la condición operativa gestionada

por la FSM de Gestión y es accesible para consulta externa.

Cuadro 22.6: Estados del subsistema definidos en *comm_sys_state_t*.

Símbolo	Valor	Significado
COMM_SYS_STATE_DOWN	0x00	Sin inicializar o detenido.
COMM_SYS_STATE_READY	0x01	Interfaz activa inicializada, recepción no habilitada.
COMM_SYS_STATE_ACTIVE	0x02	Recepción y transmisión operativas.
COMM_SYS_STATE_ERROR	0x03	Falla persistente que requiere intervención.
COMM_SYS_STATE_SELECTING	0x04	Cambiando de medio de transporte.

22.6.2 Comandos del Sistema

Los comandos constituyen el mecanismo mediante el cual los módulos externos solicitan acciones al subsistema de comunicación. Se codifican en el tipo *comm_sys_cmd_t* como una máscara de bits, lo que permite la publicación simultánea de múltiples comandos.

Cuadro 22.7: Comandos definidos en *comm_sys_cmd_t*.

Símbolo	Máscara	Acción solicitada
COMM_SYS_CMD_START	0x01	Iniciar el subsistema.
COMM_SYS_CMD_STOP	0x02	Detener el subsistema.
COMM_SYS_CMD_RESET	0x04	Reiniciar la interfaz activa sin cambiar de medio.

Los comandos se acumulan en un registro interno de la API y son consumidos en bloque por la FSM de Gestión al inicio de cada ciclo de la tarea de comunicación, garantizando que múltiples solicitudes concurrentes se procesen de forma coherente.

22.6.3 Eventos de Notificación

El subsistema de comunicación notifica de forma asíncrona a los módulos externos sobre cambios de estado, errores o transiciones de conexión. Estas notificaciones se codifican en el tipo *comm_sys_event_t*.

Cuadro 22.8: Eventos de notificación definidos en *comm_sys_event_t*.

Símbolo	Máscara	Significado
COMM_SYS_EVENT_STATE_CHANGED	0x01	El estado global del subsistema ha cambiado.
COMM_SYS_EVENT_IFACE_CHANGED	0x02	Se ha seleccionado una nueva interfaz activa.
COMM_SYS_EVENT_ERROR	0x04	Se ha producido un error crítico.
COMM_SYS_EVENT_CONNECTED	0x08	Se ha establecido una conexión física o lógica.
COMM_SYS_EVENT_DISCONNECTED	0x10	Se ha perdido la conexión.

Para recibir estas notificaciones, el módulo externo debe registrar un *callback* mediante la función *comm_sys_register_notify_callback()*.

22.6.4 Funciones de Control

`comm_sys_start()`: Publica el comando `COMM_SYS_CMD_START`. En el ciclo siguiente de la *Communication Task*, la FSM de Gestión lo consume y, si el estado actual es `DOWN`, inicia la transición a `READY`. Retorna `COMM_IFACE_OK`.

`comm_sys_stop()`: Publica el comando `COMM_SYS_CMD_STOP`. La FSM de Gestión detiene la recepción, desinicializa la interfaz activa y transita al estado `DOWN`.

`comm_sys_reset_active_iface()`: Publica el comando `COMM_SYS_CMD_RESET`. La FSM de Gestión reinicia la interfaz activa sin cambiar de medio.

`comm_sys_request_iface_change(id)`: Solicita un cambio de interfaz activa. El parámetro `id` debe corresponder a un identificador válido de `comm_iface_id_t`. La solicitud se almacena y es evaluada por la FSM de Gestión en su próximo ciclo. Si el cambio es factible, se inicia la transición a `SELECTING`. Retorna `COMM_IFACE_OK` si el identificador es válido, o `COMM_IFACE_ERROR` en caso contrario.

22.6.5 Funciones de Consulta

`comm_sys_get_state()`: Retorna el estado global actual del subsistema de comunicación.

`comm_sys_get_active_iface()`: Retorna un puntero a la estructura `comm_iface_t` de la interfaz actualmente en uso, o `NULL` si no hay ninguna activa.

`comm_sys_is_connected()`: Retorna `true` si la interfaz activa tiene una conexión física o lógica establecida.

`comm_sys_get_active_iface_info(name, id)`: Proporciona el nombre y el identificador de la interfaz activa sin exponer la estructura interna. Retorna `COMM_IFACE_ERROR` si no hay interfaz activa.

22.6.6 Notificaciones Asíncronas

`comm_sys_register_notify_callback(cb, context)`: Permite que un módulo externo registre un *callback* de tipo `comm_sys_notify_cb_t` para recibir notificaciones de eventos del subsistema. El parámetro `context` es un puntero opaco que se reenvía al *callback* en cada invocación. Solo se admite un *callback* registrado simultáneamente; una nueva llamada reemplaza al anterior.

22.7 API de Aplicación (`app_msg_iface_t`)

La API de Aplicación establece el contrato que la capa de aplicación debe implementar para recibir mensajes y notificaciones desde la capa de middleware. Esta interfaz desacopla completamente la lógica de comunicación del procesamiento del contenido: el mismo código de middleware puede servir a un analizador SCPI, a un intérprete Modbus o a cualquier otro protocolo, sin más requisito que la implementación de `app_msg_iface_t`.

22.7.1 Interfaz *app_msg_iface_t*

La interfaz se define como una estructura de punteros a función y un campo de contexto opaco. La capa de aplicación proporciona una instancia de esta estructura durante la inicialización del subsistema de comunicación.

Cuadro 22.9: *Callbacks definidos en *app_msg_iface_t*.*

Callback	Firma	Propósito
<code>on_rx_data</code>	<code>void (*)(void *ctx, const uint8_t *data, size_t len)</code>	Transferir bytes crudos desde el buffer de recepción.
<code>on_tx_done</code>	<code>app_msg_response_t (*)(void *ctx)</code>	Notificar finalización de transmisión.
<code>on_error</code>	<code>void (*)(void *ctx, app_msg_error_t error)</code>	Notificar un error de comunicación.

22.7.2 Callbacks de Recepción

`on_rx_data(ctx, data, len)`: Es invocado por la *Communication Task* en cada ciclo para transferir los bytes disponibles en el buffer de recepción. La aplicación debe procesar los datos de inmediato, ya que el puntero es válido únicamente hasta que el *callback* retorne. Esta interfaz no impone ninguna estructura de mensajes: la delimitación y el análisis sintáctico son responsabilidad exclusiva de la capa de aplicación.

22.7.3 Callbacks de Transmisión

`on_tx_done(ctx)`: Es invocado cuando la transmisión de una respuesta ha finalizado exitosamente. Permite a la aplicación liberar recursos asociados a la respuesta enviada.

22.7.4 Callbacks de Error

`on_error(ctx, error)`: Es invocado cuando la capa de middleware detecta un error que debe ser conocido por la aplicación. El parámetro `error` es una máscara de bits de tipo `app_msg_error_t` que indica la naturaleza del error.

22.7.5 Eventos de Aplicación

Los eventos que la capa de middleware puede notificar a la aplicación se codifican en el tipo `app_msg_event_t`.

22.7.6 Errores de Aplicación

Los errores que la capa de middleware puede notificar a la aplicación se codifican en el tipo `app_msg_error_t`.

Cuadro 22.10: Eventos definidos en *app_msg_event_t*.

Símbolo	Máscara	Significado
APP_MSG_EVENT_MSG_READY	0x01	Mensaje completo disponible.
APP_MSG_EVENT_TX_DONE	0x02	Transmisión completada.
APP_MSG_EVENT_RX_ERROR	0x04	Error durante la recepción.
APP_MSG_EVENT_TX_ERROR	0x08	Error durante la transmisión.
APP_MSG_EVENT_INTERNAL_ERROR	0x10	Error interno del middleware.

Cuadro 22.11: Errores definidos en *app_msg_error_t*.

Símbolo	Máscara	Significado
APP_MSG_ERROR_RX	0x01	Error durante la recepción.
APP_MSG_ERROR_TX	0x02	Error durante la transmisión.
APP_MSG_ERROR_INTERNAL	0x04	Error interno del middleware.

22.7.7 Envío de Respuestas

Para transmitir una respuesta, la capa de aplicación no accede directamente al buffer de transmisión de la *Communication Task*. En su lugar, invoca la función `comm_app_send_response(data, len)` provista por la capa de middleware.

`comm_app_send_response(data, len)`: Copia los bytes de la respuesta al buffer de transmisión de la tarea mediante `cb_put()` y activa internamente el evento lógico `EV_TX_REQUEST`. La FSM de Transmisión evalúa este evento en el ciclo siguiente e inicia el envío hardware.

22.7.8 Códigos de Retorno

Los *callbacks* de la aplicación retornan un código estandarizado de tipo `app_msg_response_t` que informa a la capa de middleware del resultado de la operación.

Cuadro 22.12: Valores definidos en *app_msg_response_t*.

Símbolo	Valor	Interpretación
APP_MSG_OK	0x00	Operación completada con éxito.
APP_MSG_ERROR	0x01	Error genérico.
APP_MSG_BUSY	0x02	Aplicación ocupada; reintentar en el ciclo siguiente.

23

Integración del intérprete SCPI con el sistema

El intérprete de comandos SCPI constituye la capa de aplicación del subsistema de comunicación. Su función es recibir mensajes desde el *middleware*, interpretarlos conforme al estándar IEEE 488.2 y ejecutar las acciones correspondientes sobre el instrumento. Este capítulo describe cómo el intérprete —basado en la biblioteca `libscpi`— se integra con las tres interfaces que el sistema ofrece: la API de mensajería con el *middleware*, la API de control del sistema y las APIs de los subsistemas de medición y control.

El capítulo no enumera los comandos SCPI disponibles, cuyo detalle se ha presentado en el capítulo anterior, sino que expone los mecanismos de conexión entre el intérprete y el resto del *firmware*.

23.1 Integración con la capa de *middleware*

La comunicación entre el *middleware* y la capa de aplicación se establece mediante la interfaz abstracta `app_msg_iface_t`, cuyo contrato se definió en el capítulo del Sistema de Comunicación. El intérprete SCPI proporciona una implementación de esta interfaz y la registra en la *Communication Task* durante la fase de inicialización del sistema.

23.1.1 Recepción de bytes crudos

El *middleware* no realiza delimitación ni análisis de los datos recibidos. En cada ciclo de la *Communication Task*, los bytes disponibles en el *buffer* circular de recepción se transfieren directamente a la aplicación mediante el *callback* `on_rx_data`. La implementación de este *callback* en el intérprete SCPI invoca la función `SCPI_Input` de `libscpi`:

```
static void on_rx_data(void *ctx, const uint8_t *data, size_t len) {  
    (void)ctx;  
    SCPI_Input(&scpi_context, (const char *)data, len);  
}
```

`libscpi` gestiona internamente su propio *buffer* de entrada y aplica las reglas de terminación configuradas. El intérprete no impone ninguna estructura de mensajes: la delimitación, el descarte de caracteres de control y el análisis sintáctico son responsabilidad exclusiva de la biblioteca.

23.1.2 Transmisión de respuestas

Cuando `libscpi` genera una respuesta (ya sea ante una consulta, un error o un comando), invoca el *callback* `write` de su interfaz. La implementación de este *callback* utiliza la función `comm_app_send_response`, que la *Communication Task* expone como servicio:

```
static size_t scpi_write_cb(scpi_t *ctx, const char *data, size_t len) {
    (void)ctx;
    comm_app_send_response((const uint8_t *)data, len);
    return len;
}
```

La transmisión de la respuesta es asíncrona: los datos se copian al *buffer* de transmisión de la tarea y la FSM de Transmisión se encarga del envío hardware. Cuando la transmisión finaliza, el *callback* `on_tx_done` notifica al intérprete, que puede liberar recursos asociados a la respuesta.

23.1.3 Notificación de errores de comunicación

Los errores detectados por el *middleware* (desbordamiento del *buffer* de recepción, fallos de transmisión) se notifican al intérprete mediante el *callback* `on_error`. La implementación actual registra el evento para su posible consulta mediante el comando `SYSTem:ERRor?`, aunque no se toma acción correctiva automática.

23.1.4 Secuencia de inicialización

Durante el arranque del sistema, el módulo `task_system` invoca `scpi_engine_init()` para configurar `libscpi` con la tabla de comandos y los *buffers* de entrada. A continuación, obtiene la interfaz `app_msg_iface_t` mediante `scpi_engine_get_iface()` y la registra en la *Communication Task* con `comm_task_register_app_iface()`. Solo después de este registro el sistema publica el comando `COMM_SYS_CMD_START`, asegurando que el intérprete esté listo para recibir datos antes de que se active la recepción.

23.2 Interacción con el sistema

El intérprete SCPI no actúa de forma aislada: los comandos que afectan al estado global del instrumento se traducen en solicitudes hacia el módulo de sistema, y el intérprete es a su vez receptor de notificaciones sobre el estado del subsistema de comunicación.

23.2.1 Comandos que afectan al estado global

Reset del instrumento (*RST). El comando *RST no modifica directamente los parámetros del instrumento. El intérprete invoca el *callback* estándar SCPI_CoreRst para la limpieza de registros IEEE 488.2 y, a continuación, publica el comando de sistema COMM_SYS_CMD_FULL_RESET mediante `comm_sys_post_command()`. La tarea de sistema (`task_system`) consume este comando en su ciclo siguiente y ejecuta `config_apply_reset_values()`, que restaura los valores por defecto de todos los canales y apaga las salidas de potencia.

Salvado y carga de configuraciones (*SAV, *RCL). Los comandos *SAV <slot> y *RCL <slot>, previstos para una implementación futura, seguirán el mismo patrón de delegación: el intérprete publicará un comando de sistema con el número de *slot* como argumento, y será `task_system` quien invoque a `config_save_to_slot()` o `config_load_from_slot()`.

Cambio de interfaz de comunicación. El comando SYSTem:COMMunicate:IFACe permite seleccionar el medio de transporte activo. El intérprete traduce la solicitud a una llamada a `comm_sys_request_iface_change()`, que la FSM de Gestión de la *Communication Task* evalúa en el ciclo siguiente.

23.2.2 Recepción de eventos del sistema

El intérprete SCPI se registra como suscriptor de las notificaciones del subsistema de comunicación mediante `comm_sys_register_notify_callback()`. Esto le permite reaccionar ante cambios de estado sin necesidad de consultas periódicas:

- **Cambio de estado:** ante COMM_SYS_EVENT_STATE_CHANGED, el intérprete puede inhabilitar temporalmente la aceptación de comandos si el subsistema no se encuentra en estado ACTIVE.
- **Error:** ante COMM_SYS_EVENT_ERROR, el error se registra en la cola interna de `libscpi` para su consulta mediante SYSTem:ERRor?.
- **Conexión / desconexión:** permiten reiniciar el estado del *parser* cuando se establece o se pierde la conexión con el *host*.

Este mecanismo convierte al intérprete en un suscriptor del sistema, aumentando la robustez sin introducir acoplamiento adicional.

23.3 Interacción con los subsistemas de control

El instrumento organiza sus recursos de medición y control en **canales**. Cada canal agrupa tres componentes —un sensor de temperatura, un controlador PID y una etapa de salida— bajo una estructura contenedora denominada `controller_t`. Esta estructura se obtiene mediante la función `controller_get_instance(channel)`, que retorna la instancia correspondiente al número de canal.

Los *callbacks* de los comandos SCPI no acceden directamente al hardware, sino que invocan las funciones definidas en las APIs de cada componente. Esto desacopla el intérprete de la implementación concreta y permite la evolución independiente de cada subsistema.

23.3.1 API de sensor

Cada sensor de temperatura expone sus servicios a través de la interfaz `sensor_api_t`.

Cuadro 23.1: Funciones de la API de sensor.

Función	Descripción
<code>read_temperature(sensor)</code>	Retorna la temperatura actual en °C.
<code>set_type(sensor, type)</code>	Configura el tipo de sensor (RTD, TC, etc.).
<code>get_type(sensor)</code>	Retorna el tipo de sensor configurado.
<code>set_protection_min(sensor, min)</code>	Establece el límite inferior de protección.
<code>get_protection_min(sensor)</code>	Retorna el límite inferior de protección.
<code>set_protection_max(sensor, max)</code>	Establece el límite superior de protección.
<code>get_protection_max(sensor)</code>	Retorna el límite superior de protección.
<code>enable_protection(sensor, enable)</code>	Activa o desactiva la protección.
<code>is_protection_enabled(sensor)</code>	Indica si la protección está habilitada.
<code>is_protection tripped(sensor)</code>	Indica si la protección se ha disparado.
<code>clear_protection(sensor)</code>	Reanuda la protección tras un disparo.

La protección del sensor opera de forma independiente al control PID. Si la temperatura medida supera el límite máximo o desciende por debajo del mínimo, la salida de potencia correspondiente se desactiva automáticamente.

23.3.2 API de controlador PID

El controlador PID proporciona las funciones de sintonía y ejecución del lazo de control. La API no incluye los límites de salida, que son responsabilidad de la etapa de potencia.

Cuadro 23.2: Funciones de la API de PID.

Función	Descripción
<code>set_setpoint(pid, sp)</code>	Establece el punto de consigna.
<code>get_setpoint(pid)</code>	Retorna el punto de consigna.
<code>set_kp(pid, kp)</code>	Establece la ganancia proporcional.
<code>get_kp(pid)</code>	Retorna la ganancia proporcional.
<code>set_ki(pid, ki)</code>	Establece la ganancia integral.
<code>get_ki(pid)</code>	Retorna la ganancia integral.
<code>set_kd(pid, kd)</code>	Establece la ganancia derivativa.
<code>get_kd(pid)</code>	Retorna la ganancia derivativa.
<code>compute(pid, input)</code>	Ejecuta una iteración del PID.
<code>reset(pid)</code>	Reinicia el estado del PID.

23.3.3 API de salida

La etapa de salida gestiona el ciclo de trabajo (*duty cycle*) y los límites que restringen su excursión. También controla la activación y desactivación de la potencia.

Cuadro 23.3: Funciones de la API de salida.

Función	Descripción
<code>set_state(out, on)</code>	Activa o desactiva la salida.
<code>get_state(out)</code>	Retorna el estado de la salida.
<code>set_limit_min(out, min)</code>	Establece el límite inferior de excursión.
<code>get_limit_min(out)</code>	Retorna el límite inferior.
<code>set_limit_max(out, max)</code>	Establece el límite superior de excursión.
<code>get_limit_max(out)</code>	Retorna el límite superior.
<code>get_duty_cycle(out)</code>	Retorna el ciclo de trabajo actual.

23.3.4 Mapeo de comandos SCPI a las APIs de control

La tabla 23.4 resume la correspondencia entre los comandos SCPI definidos para el instrumento y las funciones de las APIs que se invocan para atenderlos. En todos los casos, el número de canal se obtiene mediante `SCPI_CommandNumbers(context, &channel, 1, 1)` y la instancia del controlador se recupera con `controller_get_instance(channel)`.

23.4 Consideraciones sobre la implementación actual

En el estado actual del *firmware* (junio de 2026), las APIs de sensor, PID y salida descritas en este capítulo constituyen un modelo de referencia hacia el cual evolucionará el código. La implementación presente utiliza una única instancia de controlador PID accesible mediante `get_temp_pid_instance()` y carece de la estructura contenedora `controller_t` y de las APIs de sensor y salida como entidades independientes. Los *callbacks* del intérprete SCPI invocan directamente las funciones del módulo `pid_controller` y del ADC para la lectura de temperatura.

Esta simplificación es adecuada para la fase actual de desarrollo, que prioriza la funcionalidad sobre la abstracción. La migración hacia el modelo descrito en este capítulo se realizará de manera progresiva, sin modificar la interfaz pública del intérprete SCPI ni la de comunicación con el *middleware*.

Cuadro 23.4: Correspondencia entre comandos SCPI y APIs de control.

Comando SCPI	Función invocada
MEASure:TEMPerature#?	sensor_api_t::read_temperature
SENSe#:TEMPerature:TYPE	sensor_api_t::set_type
SENSe#:TEMPerature:TYPE?	sensor_api_t::get_type
SENSe#:TEMPerature:PROTection:MIN	sensor_api_t::set_protection_min
SENSe#:TEMPerature:PROTection:MIN?	sensor_api_t::get_protection_min
SENSe#:TEMPerature:PROTection:MAX	sensor_api_t::set_protection_max
SENSe#:TEMPerature:PROTection:MAX?	sensor_api_t::get_protection_max
SENSe#:TEMPerature:PROTection:STATe	sensor_api_t::enable_protection
SENSe#:TEMPerature:PROTection:STATe?	sensor_api_t::is_protection_enabled
SENSe#:TEMPerature:PROTection:TRIPped?	sensor_api_t::is_protection_tripped
SOURce#:CONTRol:SETPoint	pid_api_t::set_setpoint
SOURce#:CONTRol:SETPoint?	pid_api_t::get_setpoint
SOURce#:CONTRol:PID:KP	pid_api_t::set_kp
SOURce#:CONTRol:PID:KP?	pid_api_t::get_kp
SOURce#:CONTRol:PID:KI	pid_api_t::set_ki
SOURce#:CONTRol:PID:KI?	pid_api_t::get_ki
SOURce#:CONTRol:PID:KD	pid_api_t::set_kd
SOURce#:CONTRol:PID:KD?	pid_api_t::get_kd
SOURce#:CONTRol:OUTPut:LIMit:MAX	output_api_t::set_limit_max
SOURce#:CONTRol:OUTPut:LIMit:MAX?	output_api_t::get_limit_max
SOURce#:CONTRol:OUTPut:LIMit:MIN	output_api_t::set_limit_min
SOURce#:CONTRol:OUTPut:LIMit:MIN?	output_api_t::get_limit_min
SOURce#:CONTRol:OUTPut:LIMit:STATe?	output_api_t::get_duty_cycle
OUTPut#:STATe	output_api_t::set_state
OUTPut#:STATe?	output_api_t::get_state

Anexos

